DAC-203 ARTIFICIAL INTELLIGENCE · IIT ROORKEE · COURSE PROJECT

Search agents for perfect-information Generals.io

From five papers to v51

Fifty-two bots, what each one assumed, where it broke, and the model we ended with.

Built on Russell & Norvig, Artificial Intelligence: A Modern Approach (4th ed.).

OUTLINE

Nine parts

1 **The problem in AIMA's terms**
environment, game, branching

2 **The five papers, as built**
ARA*, RTAA*, depression avoidance, ProbCut, metareasoning

3 **Searching the joint move**
matrix games, ABCD, scripts

4 **Three learned components**
why each one did not pay

5 **Search from the rules**
the invariant, rollouts, plans

6 **The collection race**
D(X,T), reserve, castles

7 **Learning the opponent**
Bayes, bandits, safe exploitation

8 **v51 in depth**
fast defence and the opponent classifier

9 **What the course material bought us**
chapter by chapter

1

The problem in AIMA's terms

A two-player, zero-sum game on a 23 x 23 grid, both players moving at once.

AIMA ch. 2 (task environments), ch. 6 (adversarial search), ch. 18 (multiagent decisions)

Task environment

Property (AIMA ch. 2)	Our game
Observable	Fully: every tile, army and castle is visible
Agents	Two, competitive, zero-sum
Deterministic	Yes, after the map is drawn
Episodic / sequential	Sequential, 500 to 1,000 ticks
Static / dynamic	Dynamic: one decision per tick, 0.5 s
Discrete	Yes: 529 cells, 9 actions per cell
Known rules	Yes; we wrote an exact simulator

Production: the general and each castle add 1 every 2 ticks; every owned tile adds 1 every 50 ticks.

A move sends all but one unit, or half, to a neighbour. Combat subtracts.

The game ends when a general is taken, or at the tick cap on land, then army.

The game as a formal object

$$\mathcal{G} = (S,\ A_0,\ A_1,\ T,\ U), \qquad s_{t+1} = T(s_t,\ a_t^0,\ a_t^1), \qquad U \in \{-1,\ +1\}$$

$$b \approx 8\left|\{c:\ \mathrm{owner}(c) = p,\ \mathrm{army}(c) \geq 2\}\right| + 1, \qquad b_{\mathrm{joint}} = b_0\, b_1$$

Measured from a real mid-game position: b = 204 for one player, so about 41,600 joint actions per tick. Chess has about 35.

The reward is terminal and sparse: +1 or −1 after hundreds of ticks. Everything in between is judged by an evaluation function, so most of the work is in that function.

Simultaneous moves mean a node is a matrix game, not a max node or a min node (AIMA ch. 18). Plain minimax assumes we see the reply first.

2

The five papers, as built

Part I's plan: one paper per search chapter. v1 to v21 test them.

AIMA ch. 3 (informed search), ch. 4 (online search), ch. 6 (pruning), plus Russell & Wefald on metareasoning

ARA*: an answer at every moment

Likhachev, Gordon & Thrun, NIPS 2003

$$f(n) = g(n) + \varepsilon\, h(n), \qquad \text{cost}(\pi_\varepsilon) \;\le\; \varepsilon\, C^*$$

Built in	What happened	Why it did not decide games
v1; v33–v35	Routes found fast; time-aware A* cut expansions 31.6 → 22.6	Pathfinding was solved early; losses came from timing and home safety
Rusher test	v33, v34, v35: 3 of 6 wins each	A better route for the wrong plan is still the wrong plan

RTAA* and depression avoidance: not built

Koenig & Likhachev, AAMAS 2006 · Hernández & Baier, AAAI 2012

$$h(s) \leftarrow g(\bar{s}) + h(\bar{s}) - g(s) \qquad \forall s \in \mathrm{CLOSED}$$

$$\text{prefer } s \text{ with } h(s) = h_0(s) \quad (\text{not yet raised})$$

Assumption in the papers: true distances are unknown, so h must be learned while acting.

Our game: distances are known exactly and recomputed from the board every tick. The setting these algorithms target does not arise.

ProbCut: excellent statistics, no extra wins

Buro, ProbCut (ICCA Journal 1995) · AIMA ch. 6, forward pruning

$$v \approx a\,v' + b + e, \quad e \sim \mathcal{N}(0, \sigma^2); \qquad \text{cut if } \; v' \geq \frac{\beta + t\sigma - b}{a}$$

Measurement	Value
Fit on producer-wait positions	a = 0.955, b = 2.04, σ = 2.77
Correlation shallow vs deep	r = 0.974 (105 positions)
v17 (4-ply) vs v11, seed 68000	1–1, much slower
v20 learned correction, held-out error	2.02 → 1.79
v20 with castle-first play vs v11	lost 7 of 8

Why: pruning made the tactical tree deeper. The games were decided by economy and by when the general was left thin, over hundreds of ticks. A four-ply tree cannot see either.

Metareasoning and the move queue

Russell & Wefald, UAI 1990 · O'Ceallaigh & Ruml, SoCS 2015 · AIMA ch. 16, value of information

$$\mathrm{VOC}(c) = \mathbb{E}[U(\alpha_c)] - U(\alpha) - \mathrm{cost}(c)$$

The value of computation is high only where the best and second-best moves are close. We measured that 57% of ticks had many tied moves at one ply, so there was room for the idea.

The problem was elsewhere: extra thinking time improved decisions inside a model that was missing the thing that lost games. A bot that decides in 10 ms has nothing to gain from buying more time.

Why the paper bots were weak

Paper	What it assumes	What our game is like	Outcome
ARA*	Finding a good path is the bottleneck	Paths are easy; plans and timing are not	Kept for routing
RTAA*	Distances are unknown and learned online	Distances are exact and cheap	Not built
Depression avoidance	Heuristic basins trap the agent	No learned heuristic, no basins	Not built
ProbCut	Depth wins games	Economy and home safety win games	No gain, slower
Metareasoning	Time is scarce	10 ms per decision was enough	No gain

Common failure: all five improve search. The early bots searched a model that did not represent the event that lost games, a general left too thin when an attack arrived.

3

Searching the joint move

v3 to v35: matrix games, serialization, scripts, rollouts.

AIMA ch. 6 (alpha-beta, MCTS), ch. 18 (normal-form games, maximin, Nash)

A node is a matrix game

$$v^* = \max_{x \in \Delta_m} \min_{j} \sum_{i} x_i \, M_{ij}$$

Bot	Idea	Result
v3	Matrix game at every state	Slower and weaker than plain search
v4, v5	Matrix only when moves can collide; short safe queues	Became the baseline
v10	Two-ply matrix	Slow; lost its 4-game match against v9
v11	Adds capture of the enemy's launch source	Final bot until v37

Lesson: two stacks interact on a small fraction of ticks. The matrix matters at contact and nowhere else.

Serializing simultaneous moves: ABCD

Churchill, Saffidine & Buro, AIIDE 2012 · AIMA ch. 6, alpha-beta

$$\max_a \min_b V(a,b) \leq v^* \leq \min_b \max_a V(a,b)$$

The serialized tree is ordinary alpha-beta, so its pruning applies unchanged.

The cost is the bias: the first mover sees nothing, the second mover sees everything. With an alternating policy the two errors partly cancel across a search.

We used it at depth 2 over plans (v50). Depth 4 with 40-tick rollouts was too slow in Python: about 576 rollouts per decision before pruning.

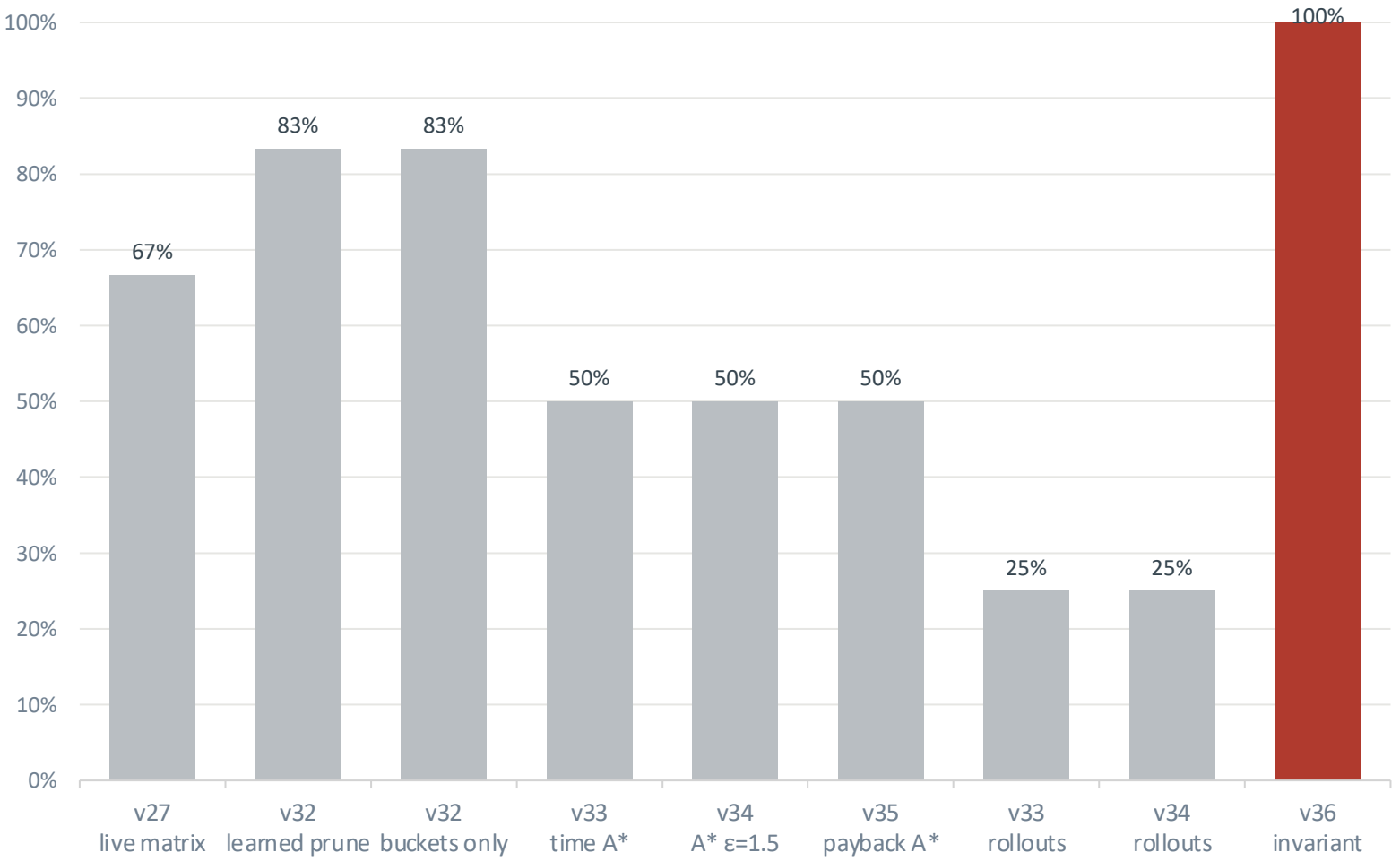
Scripts instead of moves: Puppet Search

Barriga, Stanescu & Buro, AIIDE 2015, Algorithm 1

Bot	Script search	Result
v22	Hold / castle / charge, 96 exact ticks	First temporal abstraction
v23	Adds expansion; learned macro ProbCut (error 33.3 vs 36.5)	6–6 vs v11 over 12 games
v24, v25	Long producer waits; several castle waits	Waits oscillated between targets
v33, v34	Scenario rollouts, 112 and 224 ticks	1 win, 1 draw, 2 losses each

Missing piece: nothing in these searches guaranteed that the general survived while a plan ran. A plan that wins in rollouts against four scripts can still empty the general against a fifth.

More search did not move the rusher result



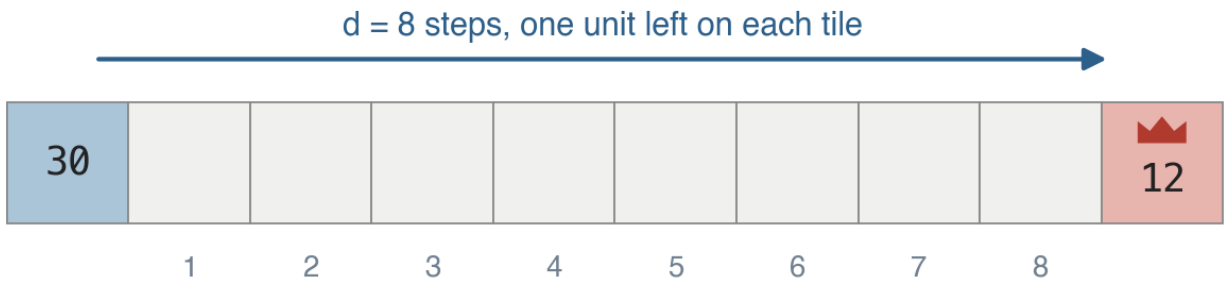
v29's second reply ply took 0.56 s on one node and still lost.

v30 lost both rusher seats; v31 took 124 s for one game and lost.

v36: 20 of 20 against the rusher and 20 of 20 against the castle rusher.

Diagnosis: the arithmetic of a rush

$$x_{\text{arrive}} = x - d - \sum_{c \in \pi} \text{garrison}(c), \qquad G(t+d) = G(t) + \lfloor d/2 \rfloor$$



$30 - 8 - 1 = 21$

$12 + \lfloor 8/2 \rfloor = 16$

margin = 16 - 21 < pad

Early wins against the rusher were land tiebreaks at short caps, not captures.

After v36 every result is reported with seats swapped and captures counted separately.

4

Three learned components

Each one learned something real. None of them won more games.

AIMA ch. 19 (learning from examples), ch. 21 (deep learning)

Why the learned parts did not pay

Bot	What was learned	Accuracy	Match result	Reason
v20	Nonlinear shallow → deep correction (ProbCut)	Error 2.02 → 1.79	Lost 7 of 8 vs v11	Better pruning of a tree that missed the cause of losses
v32	MLP that rejects optional branches	92.3% precision	5 of 6 vs rusher	Bucket-only ablation also 5 of 6: the gain came from the buckets
v40	CNN plan ranker, 8,071 regret-matched positions	Top-2 regret below random	4–16 vs v37	Noisy teacher; ranking errors compound over hundreds of ticks

AIMA's warning about learned evaluation applies: a model is only as useful as the target it imitates. All three targets were outputs of searches that did not model home safety.

5

Search from the rules

v36 and v37: an exact simulator, one invariant, then plans on top.

AIMA ch. 4 (AND-OR search), ch. 16 (decisions under uncertainty), ch. 19 (logistic regression)

An exact simulator first

Measurement	Value
Agreement with JAX step	identical boards on random actions
Recorded human moves reproduced	780 of 780
v36 decision time	1–2 ms
20 games, cap 500, 12 cores	about 20 s

Without this, every rollout in the rest of the deck would have been too slow to run inside a decision.

With it, a decision can afford a few hundred simulated ticks.

The invariant: the general must hold alone

$$m = \min_k [\, G + \text{growth}(d_k) - x_k \,] > \text{pad}, \qquad \text{pad} = 3 + 0.08\, x_{\max}$$

v36 vs	Result
Direct rusher	20 / 20
Castle rusher	20 / 20
v11 (old final bot)	12 / 12
Expander	16 / 20, losses are tiebreaks

Our own stacks are not walls: an enemy walk through our land assumes each tile holds at most 3, because stacks can step aside.

The general never relies on being refilled. Otherwise it can step out, look safe because a stack will come back, and oscillate.

Rollout checks as AND-OR search

Check	OR node (ours)	AND node (theirs)	Accept if
survives(move)	the candidate move, then the Defender script	top 3 enemy stacks × shortest and cheapest route, on the old and new board	no rollout loses the general
verify(plan)	the attack plan	Rusher, CastleRusher, Pass, turtle Econ	it wins against all four within d + 4 ticks

The checks are deliberately incomplete: the enemy policy set is small. That is the price of running them inside a 10 ms decision. Section 6 shows what this missed.

Plans, not moves (v37)

$$S(a) = \tfrac{1}{2}\sum_{b} P(b)\, V(a,b) + \tfrac{1}{2}\min_{b} V(a,b)$$

$$V(s) = \sigma\big(w_\phi^\top\ \phi(s)\big), \qquad \phi(s) \in \mathbb{R}^9$$

Measurement	Value
Plans after merging identical rollouts	6.1 → 4.6
Value model log-loss, opening	0.473 (land-only 0.522)
Value model log-loss, mid-game	0.372 (land-only 0.359)
Expander	19 / 20 (v36: 16 / 20)
vs v36 / vs v11	12–8 / 20–0

The value model did not beat a land-only baseline in the mid-game. We kept it because it helped in the opening and the end, and said so.

Phases come from the board, not the clock: opening while no enemy walk reaches us and 35% of land is neutral; end when the cap is near or one side leads 2 to 1.

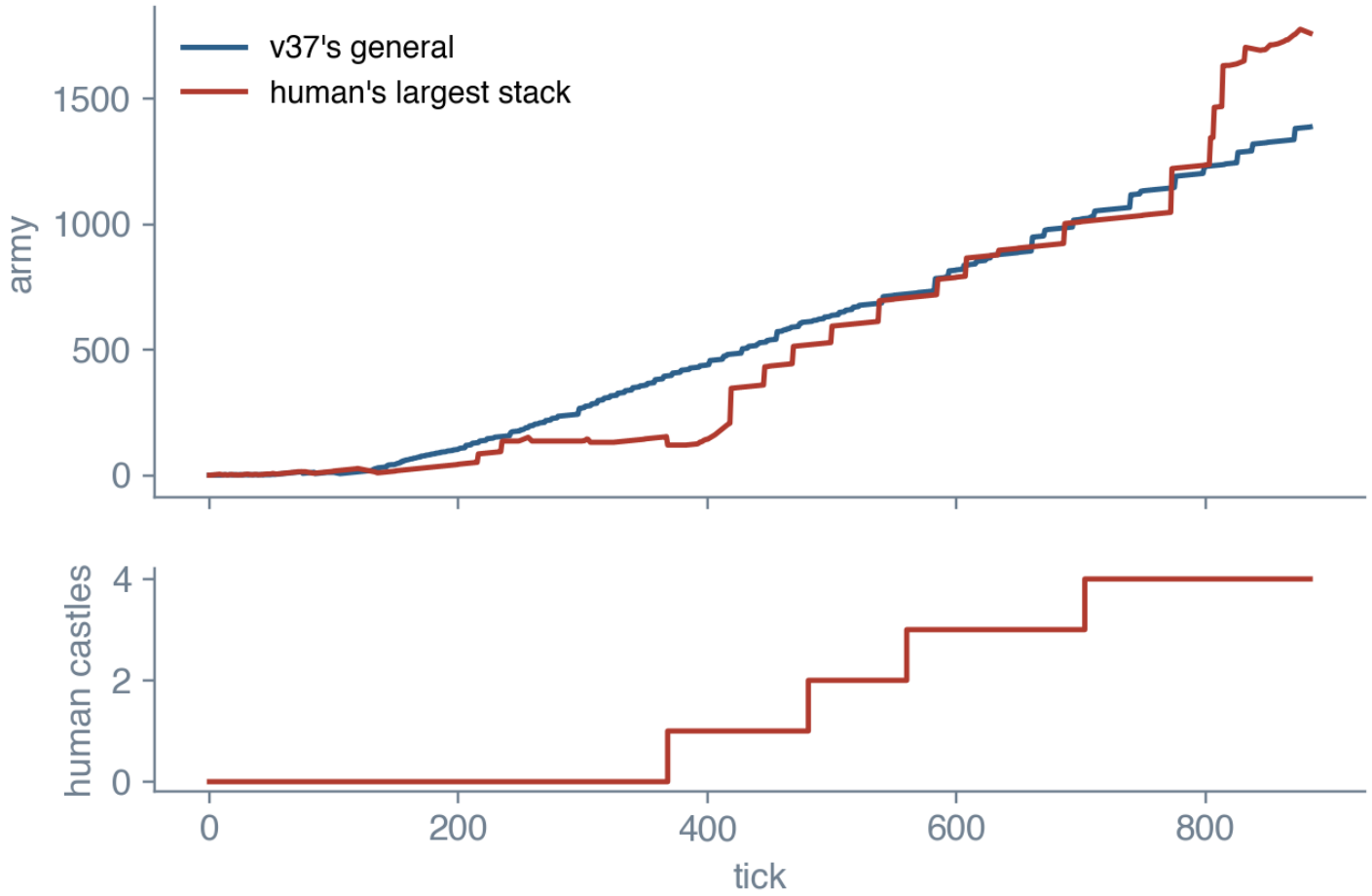
6

The collection race

v42 and v44: what people did to v37, and the margin that sees it.

Prize-collecting Steiner tree and knapsack (NP-hard), greedy approximation; AIMA ch. 5, constraints

How people beat v37



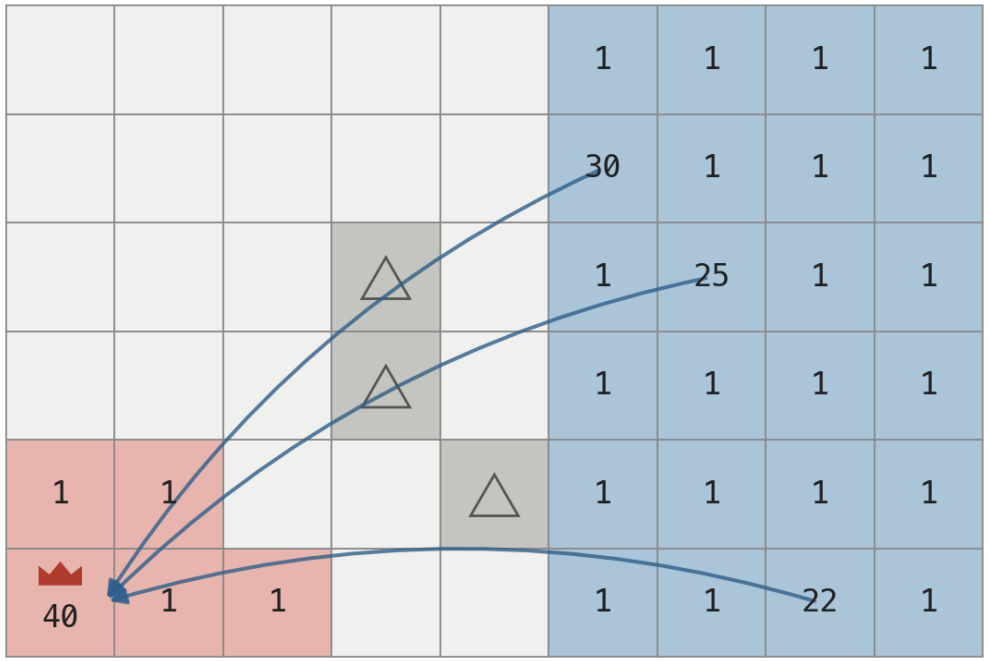
Threat model of v37: the strongest single walk.

Human strategy: many walks, collected in sequence, each one cheap.

One move per tick means a collection takes as many ticks as it has walks, which gives the defender time, if the defender is counting.

D(X, T): what a side can deliver in T moves

$$D_p(X,T) = \max_{W} \sum_{w \in W} \text{force}(w) \quad \text{s. t.} \quad \sum_{w \in W} |w| \le T \quad \text{(each tile once)}$$



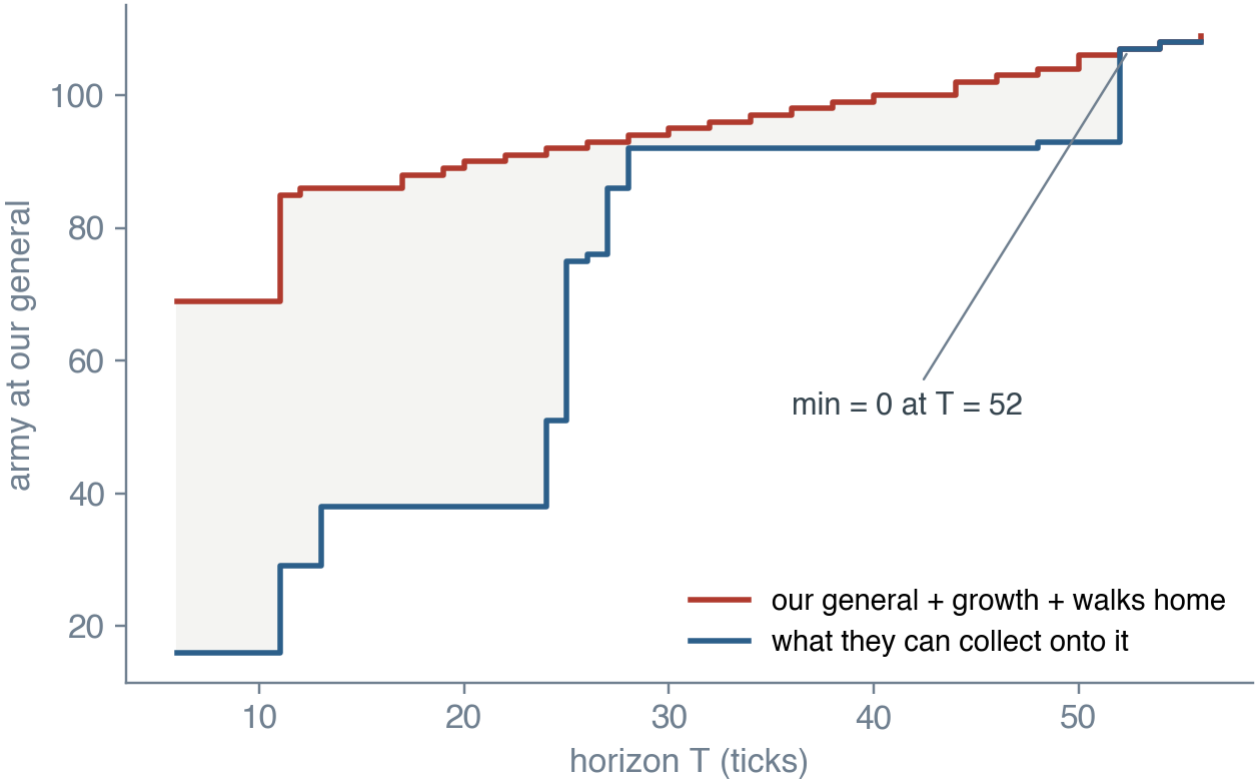
One walk at a time: the biggest arrives with 21 against a general of 40. Safe.

Three walks in sequence, 25 moves: 21 + 16 + 15 = 52 against 40 + 12 growth. Not safe.

Greedy is not optimal, but it errs toward the attacker for threats and toward caution for our own walks home.

The race margin

$$m = \min_{T} \left[G + \text{growth}(T) + D_{\text{us}}(g,\, T - 2 - \lfloor T/10 \rfloor) - D_{\text{them}}(g, T) \right]$$



T ranges over every enemy walk length and every sum of two, up to 60.

Enemy walks treat our tiles as holding at most 3. Our walks home count only through our own land, since a walk through theirs can be cut.

v42 vs v37: 19–1 on the real harness. Promoted.

Reserve and castles (v44)

$$G_{\text{keep}} + \text{growth}(H + d) \;\geq\; R \cdot x(H), \qquad H \in \{0, 30, 60\}, \quad R \in \{0.4,\ 0.6\}$$

$$2\,(\text{garrison} + d) \;\leq\; \min(L - d,\ 400)$$

v44 vs	Result
v42	13–7
v37	19–1
Five scripted bots	82 / 100 (v42: 75)

A castle produces as much as 25 plain tiles, so castles decide long games.

The same walk that buys a castle drains the general. The projection to the end of the walk is what makes both true at once.

7

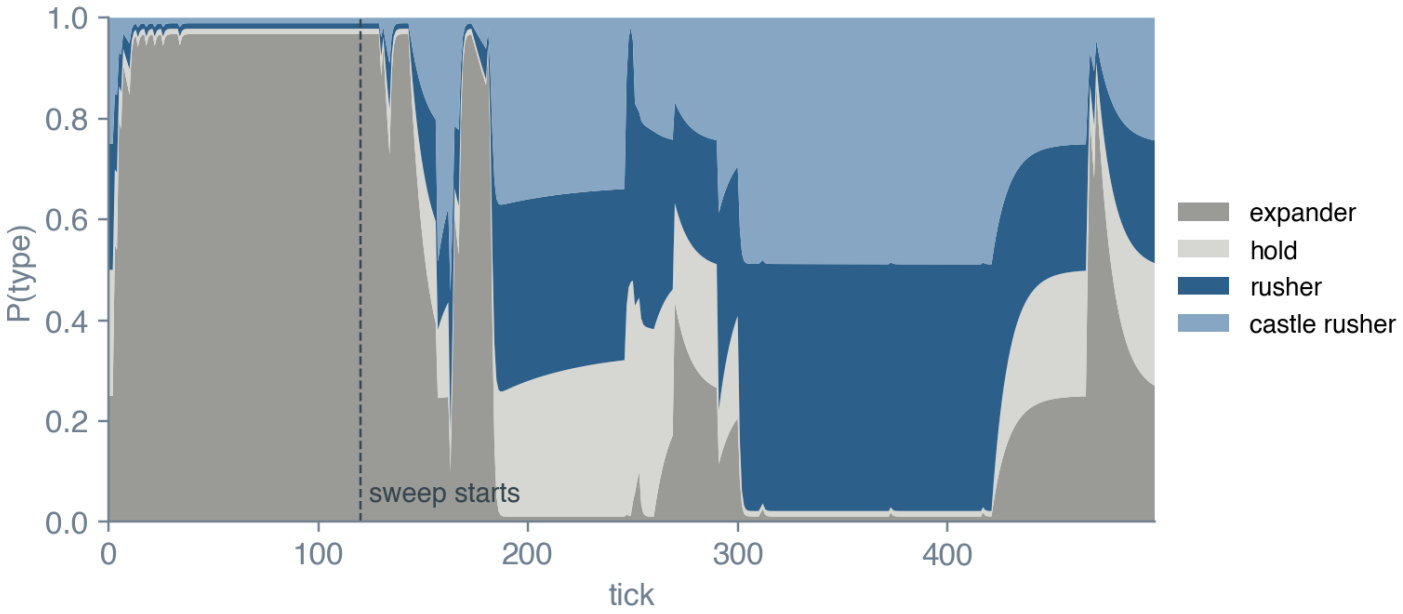
Learning the opponent

v45 to v50: belief over types, gated behaviour, safe exploitation, bandits.

AIMA ch. 12 (Bayes' rule), ch. 14 (filtering), ch. 17 (bandits), ch. 18 (game theory)

A posterior over opponent types

$$w_b \leftarrow 0.95\, w_b \frac{L_b}{0.5} + 0.05, \qquad L_b \in \{0.8,\ 0.2\}$$



Against a bot that expands to tick 120 and then sweeps its land into one charge: the belief reads expander for the whole expansion and moves to the two rusher types during the sweep.

AIMA ch. 14 calls this filtering: predict, observe, update.

Gates, safe exploitation, bandits

$$\Delta = (A_{\text{us}} - A_{\text{them}}) + (\iota_{\text{us}} - \iota_{\text{them}})\, L, \qquad \iota = \tfrac{1}{2}\,\text{producers} + \tfrac{\text{land}}{50}$$

$$a^{\dagger} = \arg\max_{a} \sum_{b} P(b) V(a,b) \quad \text{s.t.} \quad \min_{b} V(a,b) \geq v^{*} - k$$

$$p_i = (1-\gamma)\frac{w_i}{\sum_j w_j} + \frac{\gamma}{K}, \qquad w_i \leftarrow w_i\, e^{\gamma r/(p_i K)}$$

The gates can make the bot greedier or more careful. They cannot switch off the safety layer, so a wrong belief costs efficiency and never the general.

v46 vs v37 17–3, vs v42 14–6, vs v44 11–9; 81 / 100 against five scripted bots. Promoted, and hardest of our bots for people to beat at the time.

v50: the papers' actual algorithms

Algorithm (paper)	Alone vs v46
Rooted prize-collecting Steiner tree gather, bisection on λ (Straka et al. 2026)	13–7
Min-cost flow expansion + multiple-choice knapsack (Straka et al. 2026)	5–15
Local matrix game solved by regret matching (Lisý et al. 2013)	11–9
Puppet ABCD + SES (Barriga 2015; Ganzfried & Sandholm 2015)	10–10
Bayesian timing + kNN typing (Synnaeve & Bessière 2011; Weber & Mateas 2009)	13–7
Castle build order by branch and bound (Churchill & Buro 2011)	9–11
Nash arm over the meta-game (Tavares et al. 2016)	10–10
Full v50, flow expansion off, 60 games	35–25

8 v51 in depth

v46 plus a committed fast defence and a learned 10-type opponent classifier.

AIMA ch. 4 (search), ch. 14 (filtering), ch. 21 (neural networks); Straka et al. 2026 (gathering)

Architecture

fast defence + classifier (v51)
approach detector, PCST gather, 10-type MLP

opponent gates (v45, v46)
income race, posterior, Exp3 arm

collection race (v42, v44)
D(X,T) margin, reserve, castles, counter

plans (v37)
6 macros, rollouts vs 4 scripts, Hurwicz score

safety (v36)
invariant, survives(), verify(), swap-safe

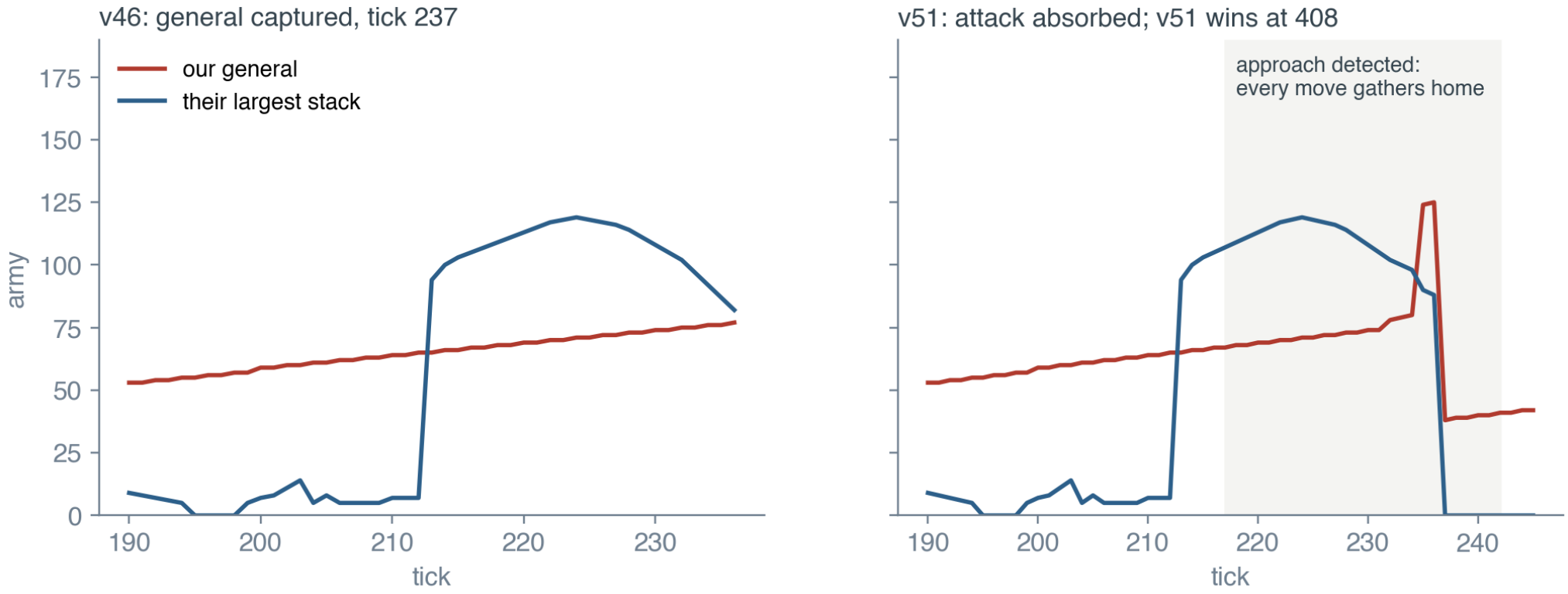
simulator (sim36)
exact replica of the game step, used for every rollout

New in v51: an approach detector, a defence-need estimate, a Steiner-tree gather sized to the attacker's arrival, and a merge rule for stacks next to the general.

Also new: a neural opponent classifier over 10 types, trained on the Mac GPU.

Everything below the top layer is v46 unchanged.

The loss that motivated it



Same map, same seat, same opponent. Left: v46. Right: v51, whose general holds 124 when the attacker arrives with 90.

Step 1: detect the approach

$$\text{incoming}(s) \Leftrightarrow d_{t-k}(s) - d_t(s) \geq 2, \quad k \leq 5, \quad x_s \geq \max(15,\, 0.2\,G)$$

Tracking: a stack keeps its history if it is on the same cell or a neighbour of last tick's cell.

Commitment: once incoming, the state lasts six ticks past the last approaching observation. v46's failure was the opposite, deciding afresh every tick and drifting back to expansion.

Threshold: 15 army or 20% of our general; 12 or 15% against collectors.

Step 2: how short will we be?

$$\text{need} = \max_{T \le \text{ETA}+5} [D_{\text{them}}(g, T) - G - \text{growth}(T)] + 0.15\, x_{\text{att}}$$

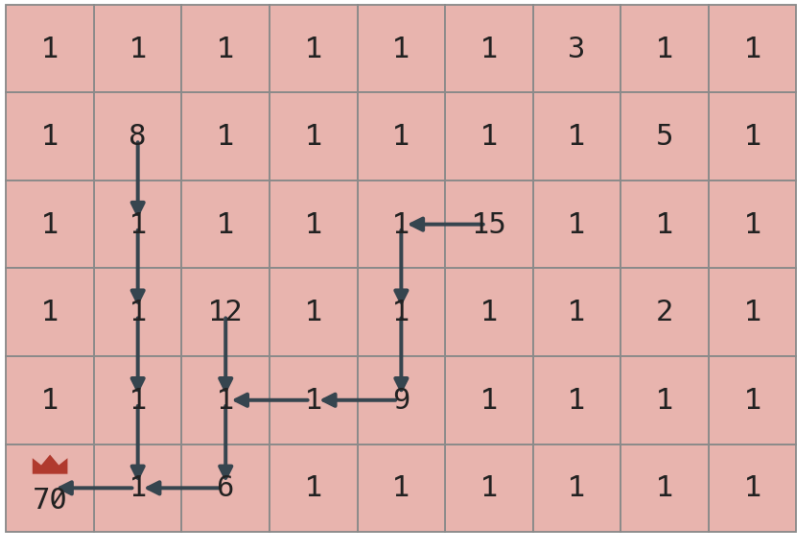
D_them is the same collection estimate as in section 6, with each tile counted once.

The need counts only the general. Stacks parked next to it do not count, because the attacker can land on the general before they move in.

The rule that follows: no stack waits beside the general.

Step 3: gather into the general with a Steiner tree

$$\max_{\mathcal{T} \ni g} \sum_{v \in \mathcal{T}} (a_v - 1) - \lambda |E(\mathcal{T})|, \qquad \lambda^* = \min\{\lambda : |E(\mathcal{T}_\lambda)| \le \mathrm{ETA} - 2\}$$



13 tree edges = 13 moves, played leaf-first into the general. The 3, 2 and 5 on the right are left out: at this lambda the moves to reach them cost more than the army they would add.

Greedy step: from the current tree, run one BFS over our land and add the path whose collected army minus lambda times its length is largest.

Bisection: 7 steps between 0 and the largest prize.

Merge rule: any stack of 10 or more next to the general steps in at once; within 6 ticks of impact, any stack of 2 or more.

Opponent classifier: features

#	Feature	#	Feature
1	tick / 500	8	change in that distance over 5 ticks
2	land / 100	9	land gained over 20 ticks
3	total army / 300	10	top stack growth over 5 ticks
4	share of army in their general	11	cells with 5 or more army
5	share of army in their top stack	12	times their general was drained
6	castles held / 3	13	army gained over 20 ticks
7	distance of top stack to our general	14	top stack within 10 tiles

Opponent classifier: model and training

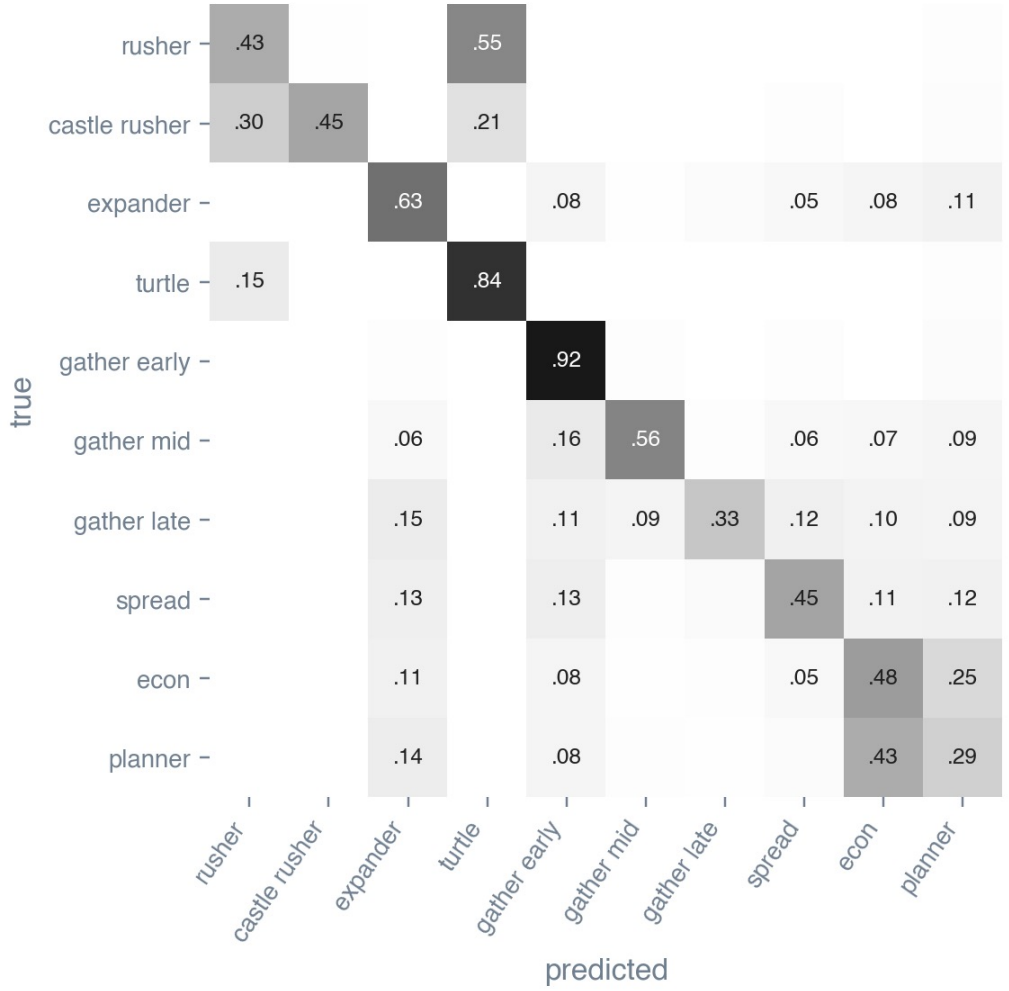
$$P(y \mid x) = \mathrm{softmax}\big(W_3\,\mathrm{ReLU}(W_2\,\mathrm{ReLU}(W_1\tilde{x} + b_1) + b_2) + b_3\big), \quad \tilde{x} = \frac{x - \mu}{\sigma}$$

Item	Value
Network	14 → 64 → 64 → 10, ReLU
Training rows	99,437 (25 maps)
Held-out rows	33,318 (8 maps)
Optimiser	Adam, lr 0.002, 60 epochs
Device	Apple GPU (MPS)

AIMA ch. 21: a small feed-forward network trained by cross-entropy.

Held-out per-tick accuracy by phase: 25% before tick 100, 53% at 100–200, 70% at 200–300, 67% after. Chance is 10%.

What the classifier confuses, and why



rusher ↔ turtle: both keep army home until the rusher launches.

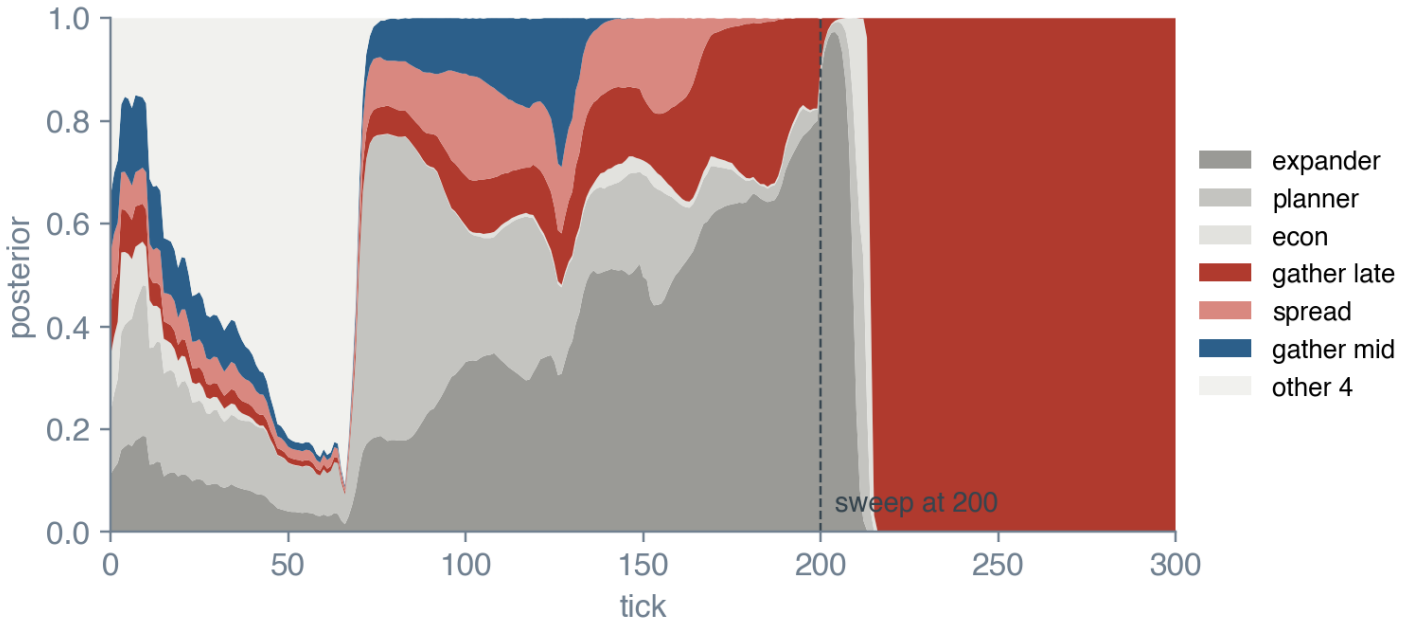
gather late ↔ expander: identical before tick 200.

econ ↔ planner: 43% of planner ticks read as econ.

The defence does not depend on these labels: the approach detector reacts to a stack that is actually walking at the general.

From per-tick guesses to a belief

$$\ell_t(y) = 0.95\,\ell_{t-1}(y) + 0.4 \log P(y \mid x_t), \qquad \hat{y}_t = \arg\max_y \ell_t(y)$$



The 10 types map onto v46's four lookahead scripts, so the planner uses them directly.

Aggressive types open the threat gate from tick 60 instead of 150.

Collector types lower the approach threshold.

One tick of v51

```
classify:  x_t ← features(board);  ℓ ← 0.95 ℓ + 0.4 log P(y | x_t)
defend:    if a stack is incoming and need > 0:
               merge a big stack next to the general, else next PCST move (budget ETA − 2)
capture:   a stack next to their general beats it → take it
counter:   home loses the race → strike first if we land sooner (verified)
urgent:    v36: verified rush, or defend when margin ≤ pad
castle:    gated castle walk, checked past its end
plan:      committed macro plan, replanned on events; every move passes safe_after()
fallback:  Defender script
```

Average decision time about 10 ms in the simulator.

Results

Opponent	Setting	v51 / v52	v46
v46 head to head	60 games, cap 500	32–28 (v51)	–
	captures in those 60 games	2 won, 2 lost	–
Gather-rusher, sweep at 200	20 games, cap 600	19–1 (v52)	12–8
Gather-rusher, sweep at 120	20 games, cap 600	20–0 (v52)	18–2
Gather-rusher, sweep at 60	20 games, cap 600	19–1 (v52)	16–4
Spread collector	20 games, cap 600	13–7 (v52)	14–6
Spread collector, late	20 games, cap 600	12–8 (v52)	14–6
Direct and castle rushers	40 games, cap 600	40 / 40	–

Limits we know about

Spread collectors are slightly harder for v51 than for v46 (13–7 and 12–8 against 14–6). They take many castles, about 4 a game, and the committed defence gives them time to do it.

The classifier has not been shown to win games: v51 and v52 both scored 32–28 against v46.

Early in the game the classifier is weak, 39% at tick 100, because early boards look alike.

Against a careful opponent v51 attacks late. A verified attack exists only when the enemy general is drained; v49 tried a guaranteed-kill search and gained little against v46.

All numbers are from 20 to 60 game samples. Differences of two or three games are noise.

9

What the course material bought us

Chapter by chapter: what we used, where it lives in v51, whether it helped.

Russell & Norvig, Artificial Intelligence: A Modern Approach, 4th edition

AIMA concepts in the final bot

AIMA idea (chapter)	Where in v51	Verdict
Task environments (2)	Fully observable, simultaneous, dynamic: framed every design choice	essential
Informed search, A* (3)	Routing; exact BFS distances	solved early
AND-OR search (4)	survives() and verify() rollouts	the safety layer
Constraints (5)	Safety margin as a hard constraint on every move	essential
Alpha-beta, pruning (6)	ABCD over plans (v50); ProbCut era (v12–v21)	little gain
Bayes' rule, filtering (12, 14)	Opponent posterior; classifier belief	useful, not decisive
Decisions, VOI (16)	Hurwicz plan score; metareasoning (v18)	score kept, VOI not
Bandits (17)	Exp3 across games	untested in matches
Game theory (18)	Matrix games, maximin, safe exploitation	local fights only
Learning, networks (19, 21)	Logistic value model; 10-type MLP	modest

Five things we learned

1 **Measure the right thing.** Swap seats, count captures apart from tiebreaks, and use caps long enough for captures to happen. Half our early gains were noise.

2 **Model the event that loses games.** Twenty bots of deeper and learned search moved nothing. One rule about the general took the rusher from 3 of 6 to 20 of 20.

3 **Constraints before objectives.** Safety as a hard constraint, preference as the objective, kept every later layer from undoing the earlier ones.

4 **People are the best test set.** Every big step after v37 came from replaying a game a person had won.

5 **Learn the opponent inside the safety layer.** Beliefs may change what v51 prefers. They never change what it is allowed to do.

Papers and book

Russell, S. & Norvig, P. Artificial Intelligence: A Modern Approach, 4th ed. Pearson, 2020.

Likhachev, M., Gordon, G. & Thrun, S. ARA*: Anytime A* with provable bounds on sub-optimality. NIPS 2003.

Koenig, S. & Likhachev, M. Real-time adaptive A*. AAMAS 2006.

Hernández, C. & Baier, J. Real-time adaptive A* with depression avoidance. AAAI 2012.

Buro, M. ProbCut: an effective selective extension of the alpha-beta algorithm. ICCA Journal, 1995.

Russell, S. & Wefald, E. Principles of metareasoning. Artificial Intelligence, 1991; UAI 1990.

Churchill, D., Saffidine, A. & Buro, M. Fast heuristic search for RTS game combat scenarios (ABCD). AIIDE 2012.

Barriga, N., Stanescu, M. & Buro, M. Puppet Search: enhancing scripted behavior by look-ahead search. AIIDE 2015.

Straka, M., Lisý, V. & Schmid, M. Superhuman AI for Generals.io using self-play reinforcement learning. arXiv 2606.23348, 2026.

Synnaeve, G. & Bessière, P. A Bayesian model for opening prediction in RTS games. AIIDE 2011.

Weber, B. & Mateas, M. A data mining approach to strategy prediction. CIG 2009.

Ganzfried, S. & Sandholm, T. Safe opponent exploitation. ACM TEAC, 2015.

Churchill, D. & Buro, M. Build order optimization in StarCraft. AIIDE 2011.

Tavares, A. et al. Rock, Paper, StarCraft: strategy selection in real-time strategy games. AIIDE 2016.

Auer, P., Cesa-Bianchi, N., Freund, Y. & Schapire, R. The nonstochastic multiarmed bandit problem. SIAM J. Comput., 2002.

Lisý, V., Kovařík, V., Lanctot, M. & Bošanský, B. Convergence of MCTS in simultaneous move games. NeurIPS 2013.